Code Smells

Refactoring

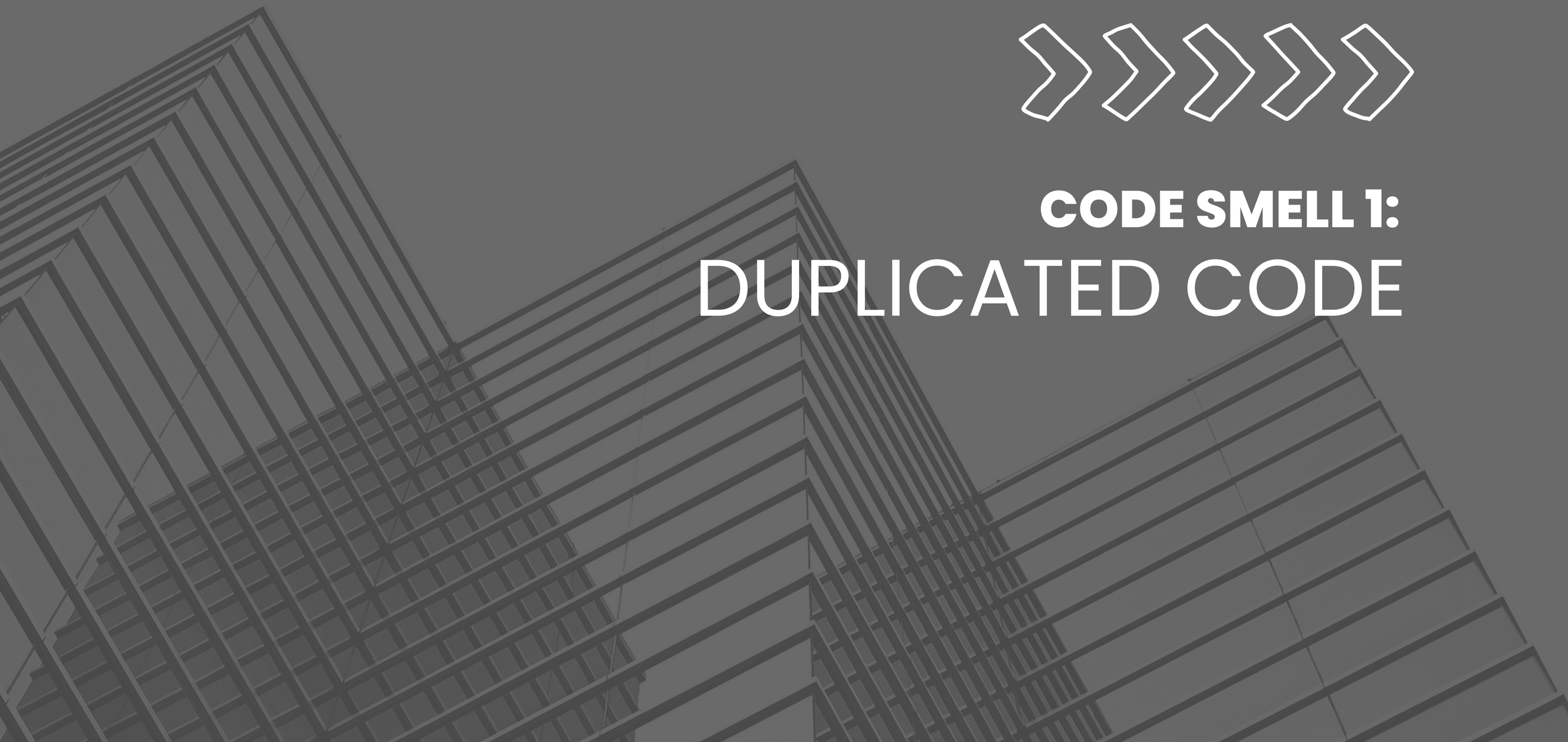
ANÁLISE DE DUPLICATED CODE, DIVERGENT CHANGE E REFUSED BEQUEST

Projeto Analisado:

Apache Tomcat
módulo - Catalina

Classe em Foco: org.apache.catalina.connector.Request.java

- **Papel da Classe:** Atua como o wrapper interno central que converte requisições de rede de baixo nível (Coyote) para a interface de aplicação Java Web (HttpServletRequest).
- **Por que analisar o Tomcat?**
  - Projeto histórico, open source e de grande escala (milhões de linhas de código).
  - Manutenção contínua ao longo de décadas para suporte a novas especificações do Jakarta EE/Java EE.
- **Acúmulo de Code Smells em Projetos Legados:**
  - **Evolução Orgânica:** O crescimento acelerado e adições frequentes de funcionalidades levam ao surgimento do Divergent Change.
  - **Evolução de Especificações:** Adaptações para manter compatibilidade com versões antigas geram Refused Bequest e Duplicated Code.



CODE SMELL 1:
DUPLICATED CODE

CODE SMELL 1:

DUPLICATED CODE

REFACTORING APLICADO: Extract Method (Extração de Método) & Iteração Dinâmica.

ORIGEM

Inicialização idêntica de parâmetros no construtor `Request()` e no método `recycle()`.

O PROBLEMA

- Violação do princípio DRY (Don't Repeat Yourself).
- **Alto risco de bugs:** adicionar um novo parâmetro no construtor exige lembrar de alterá-lo também no `recycle()`.

SOLUÇÃO OBTIDA

- Centralização da lógica de inicialização em método auxiliar privado (`copyConnectorLimits()`).
- Varredura de atributos via laço/array dinâmico, reduzindo a complexidade do código.

Duplicated Code na Classe

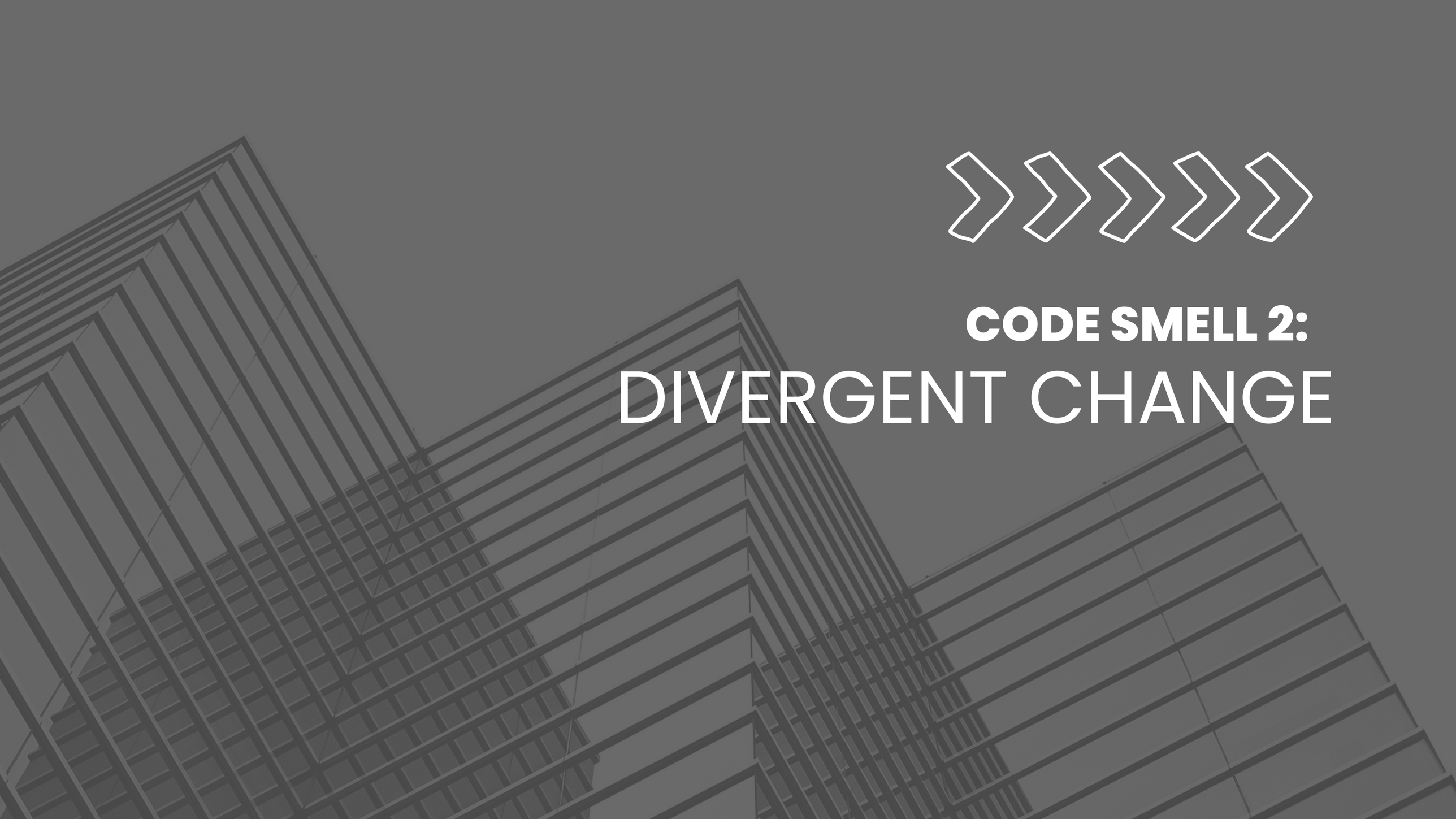
Request.java

Código Original

```
// Construtor Request()
if (connector != null) {
    maxParameterCount = connector.getMaxParameterCount();
    maxPartCount = connector.getMaxPartCount();
    maxPartHeaderSize = connector.getMaxPartHeaderSize();
    maxLocaleCount = connector.getMaxLocaleCount();
}
// Método recycle() (Duplicação idêntica)
if (connector != null) {
    maxParameterCount = connector.getMaxParameterCount();
    maxPartCount = connector.getMaxPartCount();
    maxPartHeaderSize = connector.getMaxPartHeaderSize();
    maxLocaleCount = connector.getMaxLocaleCount();
}
```

Código Refatorado

```
// Construtor e recycle() chamam o método extraído
private void copyConnectorLimits() {
    if (connector != null) {
        maxParameterCount = connector.getMaxParameterCount();
        maxPartCount = connector.getMaxPartCount();
        maxPartHeaderSize = connector.getMaxPartHeaderSize();
        maxLocaleCount = connector.getMaxLocaleCount();
    }
}
// Chamado no Construtor e no recycle():
this.copyConnectorLimits();
```



CODE SMELL 2:
DIVERGENT CHANGE

CODE SMELL 2:

DIVERGENT CHANGE

REFACTORING APLICADO: Extract Class / Princípio da Responsabilidade Única (SRP)

ORIGEM

A classe `Request.java` centraliza gestão de I/O, Segurança/TLS, Sessão HTTP, Cookies, Uploads (Multipart) e Idiomas (Locales).

O PROBLEMA

- **Violação do SRP:** Mudanças por motivos independentes (ex: alterar uploads afeta o código de segurança).
- Alto risco de efeitos colaterais e bugs em módulos não relacionados.

SOLUÇÃO OBTIDA

- **Extract Class:** Decomposição da God Class em delegados especializados (`RequestSecurityContext`, `MultipartParser`).
- A classe `Request` passa apenas a delegar chamadas aos componentes.

Divergent Change na Classe Request.java

Código Original

```
public class Request implements HttpServletRequest {
    // 1. I/O e Buffers de Rede
    protected final InputBuffer inputBuffer;
    protected CoyoteInputStream inputStream;
    // 2. Autenticação e Segurança
    protected String authType;
    protected Principal userPrincipal;
    protected transient Subject subject;
    // 3. Gestão de Sessão HTTP
    protected Session session;
    protected String requestedSessionId;
    // 4. Uploads e Formatos Multipart
    protected Collection<Part> parts;
    private int maxPartCount;
    // ... Classe com mais de 5.000 linhas alterada por
múltiplos motivos!
}
```

Código Refatorado

```
// Delegação para componentes
especializados (SRP)
public class Request implements
HttpServletRequest {
    // Delegação de responsabilidades
isoladas
    private final RequestSecurityContext
securityContext;
    private final RequestSessionManager
sessionManager;
    private final MultipartParser multipartParser;
    private final InputBuffer inputBuffer;
    // Exemplo de delegação limpa do contrato:
    @Override
    public Principal getUserPrincipal() {
        return
this.securityContext.getUserPrincipal();
    }
    public Collection<Part> getParts() {
        return this.multipartParser.parseParts();
    }
}
```



CODE SMELL 3:
REFUSED BEQUEST

CODE SMELL 3:
REFUSED BEQUEST

REFACTORING APLICADO: Composição / Padrão Decorator (Wrapper)

ORIGEM

- A classe implementa a interface pública HttpServletRequest.
- Métodos como `setRequest()` e `getAttribute()` lançam **IllegalArgumentException** se o objeto recebido não for o facade interno.

O PROBLEMA

A subclasse herda métodos mas os desativa (lançando exceções como **UnsupportedOperationException**) porque não fazem sentido para ela.

SOLUÇÃO OBTIDA

- Violação do LSP (Liskov): Recusa o contrato polimórfico da interface base.
- Promete aceitar qualquer `HttpServletRequest`, mas rejeita implementações válidas em tempo de execução.

Refused Bequest na Classe

Request.java

Código Original

```
// Recusa o contrato polimórfico genérico lançando exceção
public class Request implements HttpServletRequest {
    public void setRequest(HttpServletRequest
applicationRequest) {
        ServletRequest r = applicationRequest;
        while (r instanceof HttpServletRequestWrapper) {
            r = ((HttpServletRequestWrapper) r).getRequest();
        }
        // Recusa qualquer implementação que não seja o seu
'facade' interno!
        if (r != facade) {
            throw new IllegalArgumentException(sm.getString
("request.illegalWrap"));
        }
        this.applicationRequest = applicationRequest;
    }
}
```

Código Refatorado

```
// Uso de Composição / Decorator Pattern explícito
public class RequestFacade implements
HttpServletRequest {
    private final Request delegate;
    public RequestFacade(Request request) {
        this.delegate = request;
    }

// O acoplamento estrito fica isolado no Wrapper do servidor, sem quebrar o contrato genérico da classe Request principal.
    public void wrapRequest
(HttpServletRequestWrapper wrapper) {
        Objects.requireNonNull(wrapper, "Wrapper
não pode ser nulo");

this.delegate.setCustomWrapper(wrapper);
    }
}
```

Conclusão e considerações

Ganhos da Refatoração

- **Manutenibilidade:** Redução do acoplamento e isolamento de responsabilidades.
- **Prevenção de Bugs:** Eliminação da duplicidade de código no ciclo de vida de requisição.
- **Respeito aos Princípios SOLID:** Aplicação efetiva do SRP e LSP.

Aluna:
Mariana Barros de Moraes Sabino

Matrícula:
124211050

Disciplina:
Projeto de software

Repositório:
https://github.com/MarianaSabino-lab/Atv4-CodeSmells-Refactoring

Universidade Federal
de Campina Grande

OBRIGADA PELA ATENÇÃO